SENIOR ARCHITECT MASTERCLASS

Advanced Topics

8 Topics across 8 Sections — Teach · Tips · Pocket Notes · Interview · Code · Output

Learn + Code | Pocket Notes | Interview Prep

MCP (Model Context Protocol)

Anthropic's standard for tool integration

> Imagine every app on your phone needed its own custom cable to charge. iPhone cable, Samsung cable, Pixel cable -- a drawer full of different cables. Then USB-C came along: ONE cable for everything. MCP is USB-C for AI tools. Instead of every LLM having its own custom way to connect to tools and data sources, MCP provides ONE standard protocol that works with everything.

1. MCP Servers, Clients, and Building Integrations

📖 DEEP TEACH

```
THE PROBLEM:
Imagine you want your LLM to access 10 different tools: Google Drive, Slack, a database, a calendar, a weather API, etc. Without a standard, you write 10 custom integrations. Each one is different. Updating one breaks another.

THE FIX -- MCP:
Anthropic created MCP (Model Context Protocol) as an open standard. Think of it like a universal translator between LLMs and tools.

THREE PIECES:

1. MCP SERVER (the tool side):
Imagine a restaurant that publishes a standard menu format. Any food delivery app can read it.
An MCP server wraps a tool (Google Drive, database, API) and exposes it in a standard format. The server says: 'Here are my capabilities: I can search files, read files, and create files.'

2. MCP CLIENT (the LLM side):
Imagine the food delivery app that reads the standard menu.
The LLM (Claude, GPT, etc.) uses an MCP client to discover and call any MCP server. It asks: 'What can you do?' and gets a standard list of tools.

3. MCP HOST (the orchestrator):
Imagine the delivery platform that connects restaurants to customers.
Claude Desktop, VS Code, or your custom app acts as the host, connecting clients to servers.

HOW IT WORKS:
1. Your app discovers MCP servers: 'What tools are available?'
2. Servers respond with tool descriptions (name, parameters, what they do)
3. The LLM decides which tool to use based on the user's question
4. The client calls the server with the right parameters
5. The server executes and returns results
6. The LLM uses the results to answer the user

BUILDING AN MCP SERVER (surprisingly easy):
Python: use FastMCP library. Define functions with decorators. 20 lines of code.
TypeScript: use the official MCP SDK.

WHY THIS MATTERS:
Before MCP: 10 tools x 5 LLM platforms = 50 custom integrations.
With MCP: 10 MCP servers + 5 MCP clients = 15 standard pieces. Each tool works with every LLM automatically.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of MCP like the USB standard. Before USB, every printer, mouse, and keyboard had a different connector. USB meant any device works with any computer. MCP means any tool works with any LLM.

1. MCP is becoming THE standard. Anthropic open-sourced it. Other providers are adopting it.
2. FastMCP (Python): build a server in 20 lines. pip install fastmcp.
3. Hundreds of pre-built MCP servers already exist: Google Drive, Slack, GitHub, databases, etc.
4. Claude Desktop natively supports MCP. Add servers via config file.
5. For production: deploy MCP servers as microservices. One server per tool.
6. SECURITY: MCP servers should validate inputs and authenticate callers. Do not expose raw database access.

WAR STORY: Team built custom integrations for 8 tools with Claude. Each took 2 days = 16 days total. When they switched some tools
```

to GPT-4, they had to rebuild integrations. Rewrote as MCP servers: 3 days total. Now the same servers work with Claude, GPT-4, and their local Llama. Build once, connect to everything.

📋 POCKET NOTES

MCP = USB-C for AI tools. ONE standard protocol for all tool connections.
Server: wraps a tool, exposes standard interface (search, read, create)
Client: LLM side. Discovers and calls any MCP server.
Host: orchestrator (Claude Desktop, VS Code, custom app)
Before MCP: 10 tools x 5 LLMs = 50 custom integrations
With MCP: 10 servers + 5 clients = 15 standard pieces
FastMCP (Python): build server in 20 lines. pip install fastmcp.
Hundreds of pre-built servers exist (Drive, Slack, GitHub, DBs)
Security: validate inputs, authenticate callers.
Open standard by Anthropic. Being adopted industry-wide.

🎯 INTERVIEW PREP

SAY THIS: 'I build tool integrations as MCP servers using FastMCP. This gives a standard interface that works with any LLM client -- Claude, GPT-4, or local models. One implementation serves all platforms.'

Q: Why MCP instead of custom tool calling?
A: 'Portability. A custom OpenAI function call only works with OpenAI. An MCP server works with any MCP-compatible client. Build once, connect everywhere. Plus the ecosystem has hundreds of pre-built servers.'

Q: How do you secure MCP servers?
A: 'Authentication tokens per client, input validation on all parameters, rate limiting, and audit logging. An MCP server is an API -- apply the same security practices as any production API.'

⌨️ CODE

Python

```python
# Building an MCP Server with FastMCP (20 lines!)
# pip install fastmcp

from fastmcp import FastMCP

mcp = FastMCP("Company Knowledge Base")

@mcp.tool()
def search_docs(query: str) -> str:
    """Search company knowledge base for policies and procedures."""
    # In production: search your vector DB
    db = {"refund": "30-day refund policy. Electronics: 15 days.",
          "pricing": "Pro: $49/mo. Enterprise: $199/mo."}
    for key, val in db.items():
        if key in query.lower():
            return val
    return "No results found."

@mcp.tool()
def get_user_info(user_id: str) -> str:
    """Look up customer account information."""
    users = {"123": "Alice, Pro plan, joined 2024-01"}
    return users.get(user_id, "User not found.")

@mcp.resource("docs://policies")
def get_policies() -> str:
    """All company policies as a resource."""
    return "Refund: 30 days. Warranty: 1 year."

print("MCP Server defined with:")
print(f"  Tools: search_docs, get_user_info")
print(f"  Resources: docs://policies")
print(f"\
# Run: mcp.run() or connect via Claude Desktop")
print(f"# Any MCP client can discover and use these tools.")
print(f"# Claude: 'Search for refund policy' -> calls search_docs('refund')")
```

```
# Test locally
print(f"\
Test: {search_docs('refund')}")
print(f"Test: {get_user_info('123')}")
```

📺 **OUTPUT**

```
MCP Server defined with:
  Tools: search_docs, get_user_info
  Resources: docs://policies

# Run: mcp.run() or connect via Claude Desktop
# Any MCP client can discover and use these tools.
# Claude: 'Search for refund policy' -> calls search_docs('refund')

Test: 30-day refund policy. Electronics: 15 days.
Test: Alice, Pro plan, joined 2024-01

# 20 lines. Works with ANY MCP-compatible LLM. Build once, connect everywhere.
```

Compound AI Systems

Multiple models + retrievers + tools = one system

> Imagine building a hospital. You do not hire ONE doctor who does everything -- surgery, radiology, therapy, pharmacy. You hire SPECIALISTS and build a SYSTEM that coordinates them. A Compound AI System combines multiple specialized AI components (different models, retrievers, tools, databases) into one coherent system where each component does what it does best.

2. Multi-Component Architecture + Orchestration

📖 DEEP TEACH

```
WHAT IS A COMPOUND AI SYSTEM:
Imagine a self-driving car. It is not ONE AI model. It is:
  Camera AI (sees the road)
  LIDAR AI (measures distances)
  Path planning AI (decides where to go)
  Control AI (steers and brakes)
  All coordinated by a central system.

Same idea for LLM applications. A production customer support system might be:
  CLASSIFIER (small, fast model): categorizes the query (billing/tech/general)
  RETRIEVER (vector search): finds relevant documents
  GENERATOR (large model): writes the answer
  GUARDRAILS (safety scanner): checks the answer
  EVALUATOR (judge model): scores quality
  CACHE (semantic): stores frequent answers

Each component is the BEST tool for its job. The classifier does not need GPT-4. The generator does.

WHY NOT ONE BIG MODEL FOR EVERYTHING?
1. COST: GPT-4 for classification costs 30x more than a fine-tuned BERT classifier. Same accuracy.
2. SPEED: a small classifier responds in 20ms. GPT-4 takes 2 seconds. Users feel the difference.
3. CONTROL: each component can be updated, tested, and monitored independently.
4. RELIABILITY: if the generator fails, the classifier and retriever still work. Graceful degradation.

ORCHESTRATION:
LangGraph is the standard orchestrator. Each component is a node in the graph. Conditional edges route between them. State flows
through the system.

DESIGN PRINCIPLES:
1. SINGLE RESPONSIBILITY: each component does ONE thing well.
2. LOOSE COUPLING: components communicate via standard interfaces. Replace any component without touching others.
3. OBSERVABLE: every component reports metrics. You can see where bottlenecks and failures occur.
4. GRACEFUL DEGRADATION: if one component fails, the system degrades (slower, less accurate) but does not crash.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like building with specialized LEGO kits instead of trying to build everything from one big block. Each kit (component)
is purpose-built. They snap together through standard connectors.

1. Start simple: retriever + generator. Add components as you identify specific pain points.
2. Each component should be independently testable and deployable.
3. The ORCHESTRATOR (LangGraph) is the most important piece. It controls the flow.
4. Log the FULL pipeline trace: classifier decision, retrieved docs, generated answer, safety check. Essential for debugging.
5. Cost optimization: use the cheapest capable model for each component. Only the generator needs the expensive model.

WAR STORY: Team built a chatbot as one big GPT-4 prompt. Cost: $0.08/query. Refactored into compound system: BERT classifier
($0.0001) + BM25 retriever (free) + GPT-3.5 for simple queries ($0.001) + GPT-4 only for complex ($0.03). Average cost dropped to
$0.009/query -- 89% savings. Quality: unchanged for simple queries, slightly better for complex (because the retriever found better
context).
```

📋 POCKET NOTES

Compound AI = multiple specialized components working together.
Analogy: hospital with specialists, not one doctor for everything.
Typical components: classifier + retriever + generator + guardrails + cache
Why not one model? Cost (30x), speed (20ms vs 2s), control, reliability.
Orchestrator: LangGraph. Each component = graph node.
Principles: single responsibility, loose coupling, observable, graceful degradation.
Cost: compound system typically 80-90% cheaper than one big model.
Start simple (retriever + generator). Add components for specific pain points.
Each component independently testable and deployable.

🎯 INTERVIEW PREP

SAY THIS: 'I design compound AI systems where each component is the best tool for its job. A fine-tuned BERT classifier for routing, BM25 + dense hybrid for retrieval, GPT-4 only for complex generation. This typically reduces cost 80-90% while maintaining or improving quality.'

Q: Why not just use GPT-4 for everything?
A: 'Three reasons: cost (GPT-4 classification costs 30x more than BERT), latency (BERT: 20ms, GPT-4: 2s for the same task), and reliability (each component fails independently, enabling graceful degradation).'

Q: How do you orchestrate?
A: 'LangGraph state machine. Each component is a node with conditional routing. Full pipeline tracing for observability. Each component is independently deployable and testable.'

⌨️ CODE

Python

```python
# Compound AI System: multiple specialized components
from dataclasses import dataclass
from typing import Literal

@dataclass
class PipelineResult:
    query: str
    category: str
    model_used: str
    answer: str
    cost: float

def classifier(query: str) -> Literal["simple","complex"]:
    """Small fast model -- BERT in production, rules here"""
    complex_words = ["compare","analyze","design","explain how"]
    if any(w in query.lower() for w in complex_words): return "complex"
    return "simple"

def retriever(query: str) -> str:
    """BM25 + dense hybrid -- free/cheap"""
    docs = {"refund":"30 days standard. Electronics 15 days.",
            "price":"Pro $49/mo. Enterprise $199/mo."}
    for k,v in docs.items():
        if k in query.lower(): return v
    return "No relevant docs."

def generator(query: str, context: str, complexity: str) -> tuple:
    """Route to right model based on complexity"""
    if complexity == "simple":
        return f"Based on our docs: {context}", "gpt-3.5", 0.001
    else:
        return f"Detailed analysis: {context}", "gpt-4", 0.030

def guardrail(answer: str) -> str:
    """Safety check -- fast, cheap"""
    blocked = ["internal pricing","employee names"]
    for b in blocked:
        if b in answer.lower(): return "I cannot share that information."
    return answer
```

```
# FULL COMPOUND PIPELINE
def compound_pipeline(query: str) -> PipelineResult:
    cat = classifier(query)
    ctx = retriever(query)
    answer, model, cost = generator(query, ctx, cat)
    safe_answer = guardrail(answer)
    return PipelineResult(query, cat, model, safe_answer, cost)

print("COMPOUND AI SYSTEM:")
for q in ["What is the refund policy?","Compare Pro vs Enterprise plans in detail"]:
    r = compound_pipeline(q)
    print(f"  [{r.category:7s}] {r.model_used:7s} ${r.cost:.3f} | {r.answer[:50]}...")

print("\
  Simple: BERT(classify) + BM25(retrieve) + GPT-3.5(generate) = $0.001")
print("  Complex: BERT(classify) + BM25(retrieve) + GPT-4(generate)  = $0.030")
print("  All-GPT-4 baseline: $0.030 for EVERY query. 30x more expensive on simple.")
```

📺 **OUTPUT**

```
COMPOUND AI SYSTEM:
  [simple ] gpt-3.5 $0.001 | Based on our docs: 30 days standard. Electronics...
  [complex] gpt-4   $0.030 | Detailed analysis: Pro $49/mo. Enterprise $199/mo...

  Simple: BERT(classify) + BM25(retrieve) + GPT-3.5(generate) = $0.001
  Complex: BERT(classify) + BM25(retrieve) + GPT-4(generate)  = $0.030
  All-GPT-4 baseline: $0.030 for EVERY query. 30x more expensive on simple.
```

Knowledge Graphs + LLMs

GraphRAG, entity extraction, ontology-grounded generation

> Imagine a regular search engine finds individual pages. But what if you ask 'Who does Bob ultimately report to?' The answer requires following a CHAIN: Bob reports to Alice, Alice reports to Carol (CEO). Regular RAG finds individual chunks but cannot follow chains. Knowledge Graphs store entities (people, projects) and their CONNECTIONS, letting you follow paths through related information.

3. GraphRAG + Entity Extraction + When to Use

📖 DEEP TEACH

```
THE PROBLEM REGULAR RAG CANNOT SOLVE:
Imagine 3 separate documents:
  Doc 1: 'Bob works in the engineering department under Alice.'
  Doc 2: 'Alice is VP of Engineering, reporting to Carol.'
  Doc 3: 'Carol is the CEO of Acme Corp.'

Question: 'Who does Bob ultimately report to?'
Regular RAG finds Doc 1 (Bob -> Alice). But it does not connect Alice -> Carol -> CEO. It would need to search AGAIN for Alice,
find Doc 2, then search AGAIN for Carol.

Knowledge Graph solves this:
  Bob -> reports_to -> Alice -> reports_to -> Carol (CEO)
One traversal. The answer is Carol.

HOW GRAPHRAG WORKS:
1. ENTITY EXTRACTION: LLM reads your documents and identifies entities (people, companies, products) and relationships (works_for,
reports_to, located_in).
2. GRAPH CONSTRUCTION: store entities as nodes and relationships as edges in a graph database (Neo4j).
3. COMMUNITY DETECTION: identify clusters of related entities (the engineering team, the sales team).
4. QUERY: for questions about relationships, traverse the graph. For factual questions, use regular RAG. Combine both.

WHEN TO USE (and when NOT to):
USE when: questions require connecting information across documents, your data has clear entities and relationships, multi-hop
reasoning is needed.
DO NOT USE when: questions are simple factual lookups, documents are independent (no cross-references), cost of entity extraction
is not justified.

COST REALITY:
Entity extraction: LLM reads every document and extracts entities. For 100K documents: ~$2K one-time cost. This is significant.
Only invest if multi-hop questions are a common pattern.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of a knowledge graph like a social network map. Regular RAG gives you individual profiles. GraphRAG shows you who knows who,
who works where, and how everyone is connected.

1. GraphRAG is EXPENSIVE to build. Entity extraction costs ~$2K per 100K documents. Only invest if 20%+ of your queries need cross-
document connections.
2. Neo4j is the standard graph database. Free community edition available.
3. Microsoft's GraphRAG library: pip install graphrag. Automated entity extraction + community summarization.
4. HYBRID: regular RAG for simple questions + GraphRAG for relationship questions. Route based on query type.
5. Start with a SMALL pilot. 1000 documents. Measure: does GraphRAG answer questions that regular RAG cannot?

WAR STORY: Consulting firm had 50K project documents. Partners asked: 'Which clients have we worked with in healthcare AND
fintech?' Regular RAG could not connect across documents. GraphRAG extracted: companies, industries, project relationships. Now
traverses: Client -> worked_on -> Project -> in_industry -> Healthcare AND Fintech. Found 12 cross-industry clients that were
previously invisible.
```

📋 POCKET NOTES

```
Knowledge Graph: entities (nodes) + relationships (edges)
GraphRAG: extract entities from docs -> build graph -> traverse for multi-hop Qs
Problem it solves: 'Bob reports to Alice reports to Carol' across 3 documents
Regular RAG finds individual chunks. GraphRAG follows chains.
4 steps: entity extraction -> graph construction -> community detection -> query
When: 20%+ queries need cross-document connections
When NOT: simple factual lookups, independent documents
Cost: ~$2K per 100K documents for entity extraction
Neo4j: standard graph DB. Microsoft graphrag library.
Hybrid: regular RAG + GraphRAG routed by query type.
```

🎯 INTERVIEW PREP

SAY THIS: 'I use GraphRAG when queries require multi-hop reasoning across documents -- connecting entities through relationships that span multiple sources. I route: simple factual queries to standard RAG, relationship queries to GraphRAG.'

Q: When is GraphRAG worth the investment?
A: 'When more than 20% of queries need information connected across documents. Entity extraction costs ~$2K per 100K docs. I pilot on 1000 documents first to validate the use case.'

Q: How do you build the graph?
A: 'LLM extracts entities and relationships from each document. Store in Neo4j. Community detection clusters related entities. Microsoft graphrag library automates most of this pipeline.'

⌨️ CODE

Python

```python
# Knowledge Graph concepts
from dataclasses import dataclass
from typing import List, Tuple

@dataclass
class Entity:
    name: str
    type: str  # person, company, project

@dataclass
class Relationship:
    source: str
    relation: str  # reports_to, works_at, located_in
    target: str

class SimpleKnowledgeGraph:
    def __init__(self):
        self.entities = {}
        self.relationships = []

    def add_entity(self, name, type):
        self.entities[name] = Entity(name, type)

    def add_rel(self, source, relation, target):
        self.relationships.append(Relationship(source, relation, target))

    def traverse(self, start, relation, max_hops=5):
        """Follow a chain of relationships"""
        path = [start]
        current = start
        for _ in range(max_hops):
            found = [r.target for r in self.relationships
                     if r.source==current and r.relation==relation]
            if not found: break
            current = found[0]
            path.append(current)
        return path

kg = SimpleKnowledgeGraph()
kg.add_entity("Bob", "person")
kg.add_entity("Alice", "person")
```

```
kg.add_entity("Carol", "person")
kg.add_entity("Acme Corp", "company")

kg.add_rel("Bob", "reports_to", "Alice")
kg.add_rel("Alice", "reports_to", "Carol")
kg.add_rel("Carol", "ceo_of", "Acme Corp")

print("KNOWLEDGE GRAPH:")
for r in kg.relationships:
    print(f"  {r.source} --{r.relation}--> {r.target}")

print(f"\
Query: Who does Bob ultimately report to?")
chain = kg.traverse("Bob", "reports_to")
print(f"Chain: {' -> '.join(chain)}")
print(f"Answer: {chain[-1]} (found by traversing {len(chain)-1} hops)")
print(f"\
# Regular RAG would find Bob->Alice but miss Alice->Carol.")
```

📺 **OUTPUT**

```
KNOWLEDGE GRAPH:
  Bob --reports_to--> Alice
  Alice --reports_to--> Carol
  Carol --ceo_of--> Acme Corp

Query: Who does Bob ultimately report to?
Chain: Bob -> Alice -> Carol
Answer: Carol (found by traversing 2 hops)

# Regular RAG would find Bob->Alice but miss Alice->Carol.
# GraphRAG follows the chain across documents.
```

Long-Context Techniques

Context compression, recursive summarization, memory transformers

> Imagine you have a brilliant advisor who can only read 50 pages at a time. Your document is 500 pages. You cannot hand them all 500 pages. You need techniques to make those 500 pages FIT into 50 pages without losing the important information. Long-context techniques solve this: compression, summarization, and smarter memory.

4. Compression + Recursive Summarization + Memory

📖 DEEP TEACH

```
THE PROBLEM:
Even though modern models have large context windows (128K tokens for GPT-4o, 200K for Claude), real-world scenarios often exceed
them. A codebase might be 1 million tokens. A year of customer support logs: 50 million tokens. A legal discovery set: billions of
tokens.

3 TECHNIQUES:

1. CONTEXT COMPRESSION (squeeze more into less space):
Imagine packing for a trip. Instead of bringing full outfits, you bring versatile pieces that mix and match.

LLMLingua / LongLLMLingua: compress prompts by removing redundant words while preserving meaning. A 10,000-word context might
compress to 3,000 words with 90% information retention.

How: a small model identifies which tokens are important (high information content) and which are filler (low information). Remove
the filler.

2. RECURSIVE SUMMARIZATION (the pyramid approach):
Imagine summarizing a book. You do not read all 500 pages and write one summary. You:
  Level 1: summarize each chapter (50 chapters -> 50 summaries)
  Level 2: summarize groups of summaries (50 -> 10)
  Level 3: summarize the 10 into a final summary

Each level condenses further. The final summary fits in a single context window.

This is MapReduce applied to summarization. Works for any document size.

3. MEMORY-AUGMENTED ARCHITECTURES:
Imagine a student who takes notes while reading. They cannot remember everything, but their notes capture the key points. When they
need a detail, they check their notes.

Memory approaches:
  SLIDING WINDOW: process document in overlapping windows. Each window sees the previous window's summary.
  MEMORY TOKENS: special tokens that carry compressed information from earlier in the document.
  RETRIEVAL-AUGMENTED: instead of fitting everything in context, retrieve only relevant sections per question.

THE PRACTICAL HIERARCHY (what to try first):
1. Does it fit in the context window? Just use it directly. Simplest.
2. Too big? Retrieve only relevant chunks (RAG). No need to process the whole thing.
3. Need to process the whole thing? Recursive summarization (MapReduce).
4. Need to preserve more detail? Context compression (LLMLingua).
5. Real-time processing of streams? Sliding window with memory.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of these techniques like different ways to study for an exam with too much material:
  Context compression = highlighting only key sentences in the textbook
  Recursive summarization = making summaries of summaries of summaries
  RAG = only opening the textbook to the relevant page for each question

1. TRY RAG FIRST. 90% of the time, you do not need to process the whole document. Just retrieve relevant chunks.
2. 128K tokens is A LOT. GPT-4o can handle ~300 pages. Test if your document actually exceeds the limit before adding complexity.
3. Recursive summarization: MapReduce pattern. Cheap model for MAP (summarize each chunk). Smart model for REDUCE (combine
summaries).
```

4. LLMLingua: up to 20x compression with ~90% quality retention. pip install llmlingua.
5. LOST IN THE MIDDLE: models pay more attention to the beginning and end of the context. Put important information there, not in the middle.

WAR STORY: Legal team needed to analyze 2000-page contracts. Context window: 128K tokens (~300 pages). Solution: recursive summarization to create 10-page summaries, then detailed analysis on specific sections via RAG. The summary caught all major issues. RAG answered specific clause questions. Neither technique alone was sufficient.

📋 POCKET NOTES

3 Techniques for exceeding context windows:
  Compression (LLMLingua): remove filler words. 10K -> 3K words. 90% quality.
  Recursive Summarization: summarize chapters -> summarize summaries -> final.
  Memory: sliding window, memory tokens, retrieval-augmented.
Practical hierarchy:
  1. Fits in window? Use directly.
  2. Too big? RAG (retrieve relevant chunks only).
  3. Need whole doc? Recursive summarization (MapReduce).
  4. Need detail? Compression (LLMLingua).
Lost in the middle: put important info at START and END of context.
128K tokens = ~300 pages. Test if you actually exceed it.
pip install llmlingua

🎯 INTERVIEW PREP

SAY THIS: 'I follow a practical hierarchy: direct context if it fits, RAG for targeted retrieval, recursive summarization for full-document processing, and compression for maximizing information density. 90% of the time, RAG makes the long-context problem disappear.'

Q: 2000-page legal document. How do you process it?
A: 'Two-pass: recursive summarization for a 10-page overview that catches all major issues, then RAG for specific clause-level questions. Neither alone is sufficient -- the summary gives breadth, RAG gives depth.'

Q: Lost in the middle -- what is it and how do you handle it?
A: 'Models attend more to the beginning and end of context, less to the middle. I put the most important information at the start and end. For retrieved chunks, I order by relevance with the best match first.'

⌨ CODE

Python

```python
# Long-context techniques

# 1. RECURSIVE SUMMARIZATION (MapReduce)
def recursive_summarize(chunks, summarize_fn, group_size=5):
    """Summarize in layers until it fits in one context"""
    level = 0
    current = chunks
    while len(current) > 1:
        level += 1
        groups = [current[i:i+group_size] for i in range(0, len(current), group_size)]
        current = [summarize_fn(" ".join(g)) for g in groups]
        print(f"  Level {level}: {len(groups)} groups -> {len(current)} summaries")
    return current[0]

def mock_summarize(text):
    words = text.split()
    return " ".join(words[:max(5, len(words)//3)])  # Keep first third

print("RECURSIVE SUMMARIZATION:")
chunks = [f"Chapter {i}: content about topic {i} with many details" for i in range(20)]
result = recursive_summarize(chunks, mock_summarize)
print(f"  20 chapters -> 1 summary: {result[:60]}...")

# 2. CONTEXT COMPRESSION
def compress_context(text, keep_ratio=0.3):
    """Simplified compression: keep important sentences"""
    sentences = text.split(". ")
    # In production: LLMLingua scores each token's importance
```

```
    kept = sentences[:max(1, int(len(sentences) * keep_ratio))]
    compressed = ". ".join(kept)
    print(f"  Original: {len(text)} chars | Compressed: {len(compressed)} chars")
    print(f"  Ratio: {len(compressed)/len(text)*100:.0f}% (kept {len(kept)}/{len(sentences)} sentences)")
    return compressed

print("\
CONTEXT COMPRESSION:")
long_text = "The refund policy is 30 days. This applies to all standard items. Electronics have a shorter window. The electronics
window is 15 days. All items must be unused. Original receipt is required. Damaged items are handled through warranty."
compressed = compress_context(long_text)

# 3. PRACTICAL HIERARCHY
print("\
DECISION TREE:")
for doc_size, solution in [
    ("50 pages", "Direct: fits in 128K context window"),
    ("500 pages", "RAG: retrieve only relevant chunks per question"),
    ("2000 pages", "Recursive summarization + RAG for specifics"),
    ("50K pages", "MapReduce summarization + chunked RAG")]:
    print(f"  {doc_size:12s} -> {solution}")
```

📺 **OUTPUT**

```
RECURSIVE SUMMARIZATION:
  Level 1: 4 groups -> 4 summaries
  Level 2: 1 groups -> 1 summaries
  20 chapters -> 1 summary: Chapter 0: content about topic 0...

CONTEXT COMPRESSION:
  Original: 267 chars | Compressed: 84 chars
  Ratio: 31% (kept 2/7 sentences)

DECISION TREE:
  50 pages     -> Direct: fits in 128K context window
  500 pages    -> RAG: retrieve only relevant chunks per question
  2000 pages   -> Recursive summarization + RAG for specifics
  50K pages    -> MapReduce summarization + chunked RAG
```

Structured Generation + Distillation

Constrained decoding, training small models from big ones

Two power techniques. Structured Generation = putting train tracks under the LLM so it can ONLY produce valid output (like forcing it to produce valid JSON, not 'Sure! Here is some JSON...'). Distillation = having a genius teacher (GPT-4) teach a student (small model) until the student is almost as good but 10x cheaper to run.

5. Constrained Decoding + Knowledge Distillation

📖 DEEP TEACH

```
STRUCTURED GENERATION (train tracks for the LLM):

Imagine a painter who is incredibly talented but sometimes paints outside the canvas. Constrained decoding puts a frame around the
canvas -- the painter can only paint INSIDE the frame.

THREE LEVELS:
1. JSON MODE (loose frame): the LLM must produce valid JSON syntax. But it might use wrong keys or types. {"name": 123} is valid
JSON but wrong type.

2. INSTRUCTOR + PYDANTIC (tight frame): you define EXACTLY what fields, types, and valid values. The LLM MUST produce output
matching your schema. If it fails, auto-retry.

3. OUTLINES / GUIDANCE (train tracks): controls WHICH TOKENS the model is allowed to generate at each step. The LLM literally
CANNOT produce invalid output. Like a typewriter that only has the allowed keys.

Outlines is FASTER than unconstrained generation because fewer tokens are valid at each step = less computation.
But: only works with local models (you need access to the token generation process).

KNOWLEDGE DISTILLATION (teach a student from a genius):

Imagine a master chef (GPT-4) who charges $1000/hour. You cannot afford them for every meal. So you have the master chef cook
10,000 meals while a student chef watches and learns. After training, the student chef makes 90% as good meals at $10/hour.

Process:
1. Run GPT-4 on 10,000 examples. Save the outputs.
2. Fine-tune a small model (Llama-3-8B) on those GPT-4 outputs.
3. The small model learns to mimic GPT-4's behavior.
4. Deploy the small model at 10-50x lower cost.

Quality: typically 85-95% of the teacher model.
Cost: 10-50x cheaper per inference.

WHEN TO DISTILL:
- High-volume task (100K+ calls/day)
- Task is well-defined (classification, extraction, formatting)
- 85-95% of teacher quality is acceptable

WHEN NOT TO:
- Creative or reasoning-heavy tasks (quality gap too large)
- Low volume (cost savings do not justify the training investment)
- Task changes frequently (need to re-distill every time)
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Structured generation and distillation are the two techniques that separate hobby projects from production systems.

1. INSTRUCTOR is the industry standard for structured output. pip install instructor.
2. Outlines for local models: guaranteed valid output + faster generation. pip install outlines.
3. Distillation: generate training data with GPT-4, fine-tune a small model. The 10K-example sweet spot.
4. LEGAL NOTE: check the LLM provider's terms of service. Some prohibit using their output to train competing models.
5. Distillation + quantization = double savings. Distill to 8B model, quantize to 4-bit = 100x cheaper than GPT-4.

WAR STORY: Company used GPT-4 for email classification. 500K emails/day. Cost: $15K/day. Distilled: GPT-4 classified 50K training
```

examples. Fine-tuned Llama-3-8B on those. Accuracy: 94% (GPT-4 was 97%). Cost: $300/day. 50x savings. The 3% accuracy gap was acceptable for email routing.

📋 POCKET NOTES

```
Structured Generation (3 levels):
  JSON mode: valid syntax, no schema guarantee.
  Instructor + Pydantic: exact schema + auto-retry. INDUSTRY STANDARD.
  Outlines/Guidance: token-level control. Guaranteed. Local models only. Faster.
Distillation:
  GPT-4 generates 10K examples -> fine-tune small model -> deploy at 10-50x less cost.
  Quality: 85-95% of teacher. Cost: 10-50x cheaper.
  When: high volume (100K+/day), well-defined task, quality gap acceptable.
  When NOT: creative tasks, low volume, frequently changing tasks.
Legal: check ToS for using outputs to train competing models.
Combo: distill + quantize = 100x cheaper than GPT-4.
```

🎯 INTERVIEW PREP

SAY THIS: 'I use instructor for production structured output -- Pydantic schemas with auto-retry. For local models, Outlines gives token-level guarantees. For high-volume tasks, I distill GPT-4 into smaller models to cut inference cost 10-50x while retaining 90%+ quality.'

Q: How do you decide what to distill?
A: 'Three criteria: volume above 100K calls/day (cost savings justify training), well-defined task (classification, extraction -- not open-ended creativity), and 85-95% quality is acceptable for the use case.'

Q: Is distillation legal?
A: 'Depends on the provider ToS. OpenAI allows it for certain models. Anthropic has specific terms. Always check. Alternatively, distill from open-source teachers (Llama-70B teaching Llama-8B) to avoid any legal concerns.'

⌨️ CODE

Python

```python
# Structured Generation with instructor
import instructor
from openai import OpenAI
from pydantic import BaseModel, Field
from typing import Literal, List

class EmailClassification(BaseModel):
    category: Literal["billing","technical","general","spam"]
    priority: Literal["high","medium","low"]
    summary: str = Field(description="One sentence summary")

client = instructor.from_openai(OpenAI())
result = client.chat.completions.create(
    model="gpt-4",
    response_model=EmailClassification,
    messages=[{"role":"user","content":
        "Subject: URGENT cannot login. Body: My account is locked and I have a demo in 1 hour!"}]
)
print(f"Category: {result.category}")
print(f"Priority: {result.priority}")
print(f"Summary: {result.summary}")
print(f"# Guaranteed valid schema. No parsing errors. Ever.\
")

# Distillation concept
print("DISTILLATION PIPELINE:")
print("  1. GPT-4 classifies 10,000 emails (teacher) -> $300")
print("  2. Fine-tune Llama-3-8B on those 10K examples -> $50")
print("  3. Deploy Llama-3-8B for production -> $300/day")
print("  vs GPT-4 in production -> $15,000/day")
print("  Savings: 50x. Quality: 94% vs 97%.")
print("  Break-even: 1 day. ROI after 1 month: $440K saved.")
```

📺 **OUTPUT**

```
Category: technical
Priority: high
Summary: User account locked with urgent demo deadline.
# Guaranteed valid schema. No parsing errors. Ever.

DISTILLATION PIPELINE:
  1. GPT-4 classifies 10,000 emails (teacher) -> $300
  2. Fine-tune Llama-3-8B on those 10K examples -> $50
  3. Deploy Llama-3-8B for production -> $300/day
  vs GPT-4 in production -> $15,000/day
  Savings: 50x. Quality: 94% vs 97%.
  Break-even: 1 day. ROI after 1 month: $440K saved.
```

Mixture of Experts (MoE)

Mixtral, sparse routing, efficiency

> Imagine a law firm with 8 specialized lawyers. For each case, only 2 lawyers are needed -- the tax lawyer and the contract lawyer for a business deal, or the criminal lawyer and the civil rights lawyer for a different case. You have the EXPERTISE of 8 lawyers but only PAY 2 at a time. MoE models work the same way: 8 expert sub-networks, but only 2 are activated per token.

6. How MoE Works + Mixtral + Practical Implications

📖 DEEP TEACH

```
THE PROBLEM:
Bigger models are smarter. But bigger models are also slower and more expensive. A 70B model is great but costs 10x more to run
than a 7B model.

THE FIX -- MIXTURE OF EXPERTS:
Instead of one BIG model, build 8 SMALL expert models and a ROUTER that picks which 2 experts to use for each word.

Mixtral 8x7B:
  8 expert sub-networks, each 7B parameters
  Total parameters: 47B (shared layers + 8 experts)
  ACTIVE parameters per token: only 13B (shared + 2 experts)

So you get the KNOWLEDGE of a 47B model at the SPEED of a 13B model. Like having 8 specialized lawyers but only 2 in the room at
any time.

HOW THE ROUTER WORKS:
Imagine a receptionist at the law firm. A client walks in and describes their case. The receptionist says: 'You need the tax expert
and the contract expert for this one.'

The router is a small neural network that looks at each token and picks the top 2 experts. Different tokens might use different
experts. A math token might activate the 'math expert' and 'logic expert.' A creative writing token might activate the 'language
expert' and 'style expert.'

PRACTICAL IMPLICATIONS FOR ARCHITECTS:
1. MEMORY: you need ALL 8 experts loaded in memory (47B parameters) even though only 2 are active. High VRAM requirement.
2. SPEED: inference is fast (only 2 experts compute per token). Throughput is great.
3. QUALITY: comparable to dense models of similar total parameter count.
4. POPULAR MODELS: Mixtral 8x7B, Mixtral 8x22B, Grok-1, DBRX, DeepSeek-V2.

MoE models are the BEST VALUE for self-hosting. You get big-model quality at small-model speed.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of MoE like a hospital with specialized departments. The whole hospital is big (all departments), but each patient only
visits 2 departments. The patient gets specialist care at general-practitioner speed.

1. Mixtral 8x7B is the most popular MoE model. Great quality, fast inference.
2. MEMORY: need 47B parameters loaded but only 13B compute per token. High VRAM, fast speed.
3. MoE models are IDEAL for vLLM deployment. vLLM handles the expert routing efficiently.
4. Fine-tuning MoE is harder than dense models. Expert routing can become unbalanced.
5. For self-hosting: MoE gives the best quality-per-dollar. Mixtral 8x7B on 2 GPUs matches much larger dense models.
```

📋 POCKET NOTES

```
MoE: 8 expert sub-networks + router. Only 2 experts active per token.
Mixtral 8x7B: 47B total params, 13B active. Big knowledge, small speed cost.
Router: small network picks top 2 experts per token.
Memory: ALL experts loaded (47B). Speed: only 2 compute (13B).
Best value for self-hosting: big-model quality at small-model speed.
Models: Mixtral 8x7B, 8x22B, DeepSeek-V2, DBRX.
```

Fine-tuning MoE: harder (expert routing can unbalance).
vLLM: handles MoE routing efficiently.

🎯 INTERVIEW PREP

SAY THIS: 'MoE models like Mixtral give the knowledge of a 47B model at the inference cost of a 13B model. The router activates only 2 of 8 experts per token. For self-hosting, this is the best quality-per-dollar ratio.'

Q: Why not just use a 13B dense model?
A: 'A dense 13B model has 13B of knowledge. Mixtral has 47B of knowledge but only uses 13B of compute. The MoE model is significantly smarter at the same speed. Different experts specialize in different types of content.'

Q: Downsides of MoE?
A: 'Memory: all experts must be loaded (47B for Mixtral). Fine-tuning is harder because expert balance can degrade. And the routing adds slight overhead.'

⌨️ CODE

Python

```python
# MoE model deployment with vLLM
print("MIXTURE OF EXPERTS - MIXTRAL 8x7B:")
print()
print("Architecture:")
print("  8 expert sub-networks (each ~7B parameters)")
print("  1 router network (picks top 2 experts per token)")
print("  Shared attention layers")
print()

total_params = 47  # billion
active_params = 13  # billion
experts = 8
active_experts = 2

print(f"  Total parameters:  {total_params}B (all experts loaded)")
print(f"  Active per token:  {active_params}B (only {active_experts}/{experts} experts)")
print(f"  Speed advantage:   {total_params/active_params:.1f}x less compute than dense {total_params}B")
print(f"  Quality:           Comparable to dense ~{total_params}B model")
print()

# vLLM deployment
print("Deploy with vLLM:")
print("  python -m vllm.entrypoints.openai.api_server \\\\")
print("    --model mistralai/Mixtral-8x7B-Instruct-v0.1 \\\\")
print("    --tensor-parallel-size 2  # 2 GPUs for 47B params")
print()

# Cost comparison
print("COST COMPARISON (self-hosted):")
for model, gpus, quality in [
    ("Llama-3-13B (dense)", 1, "Good"),
    ("Mixtral 8x7B (MoE)", 2, "Great (47B knowledge)"),
    ("Llama-3-70B (dense)", 4, "Great")]:
    cost = gpus * 2.0  # $2/hr per GPU
    print(f"  {model:30s} | {gpus} GPU(s) ${cost:.0f}/hr | Quality: {quality}")
print("\
  Mixtral: 2 GPUs for near-70B quality. Best value.")
```

📺 OUTPUT

```
MIXTURE OF EXPERTS - MIXTRAL 8x7B:

Architecture:
  8 expert sub-networks (each ~7B parameters)
  1 router network (picks top 2 experts per token)
  Shared attention layers

  Total parameters:  47B (all experts loaded)
  Active per token:  13B (only 2/8 experts)
```

```
  Speed advantage:   3.6x less compute than dense 47B
  Quality:           Comparable to dense ~47B model

COST COMPARISON (self-hosted):
  Llama-3-13B (dense)          | 1 GPU(s) $2/hr | Quality: Good
  Mixtral 8x7B (MoE)           | 2 GPU(s) $4/hr | Quality: Great (47B knowledge)
  Llama-3-70B (dense)          | 4 GPU(s) $8/hr | Quality: Great

  Mixtral: 2 GPUs for near-70B quality. Best value.
```

Local/Edge LLMs

Ollama, llama.cpp, MLX for Apple Silicon

Imagine being able to run a smart AI assistant on your laptop with NO internet connection. No API keys. No monthly bills. No data leaving your computer. Local LLMs make this real. They are smaller and less capable than cloud models, but they are free, private, and always available.

7. Ollama + llama.cpp + MLX + When to Use

📖 DEEP TEACH

```
3 TOOLS FOR RUNNING LLMS LOCALLY:

1. OLLAMA (the one-command solution):
Imagine installing an app and running it. That is it.
  Install: download from ollama.com
  Run: ollama run llama3
  That is literally it. The model downloads and starts chatting.

Ollama manages models like Docker manages containers. Download, run, stop, delete.
Supports: Llama, Mistral, Phi, Gemma, and 50+ models.
Expose as API: ollama serve -> OpenAI-compatible API on localhost.

2. LLAMA.CPP (the efficiency champion):
Imagine squeezing a powerful engine into a tiny car. llama.cpp is written in pure C++ for maximum efficiency. Runs on CPU (no GPU
needed!). Also supports Apple Silicon Metal GPU.

Uses GGUF format: models quantized to 4-bit or 8-bit for small size.
A 7B model: runs on 8GB RAM laptop.
A 70B model: runs on Mac with 64GB RAM.

3. MLX (Apple Silicon native):
Imagine a framework built SPECIFICALLY for your Mac's M1/M2/M3/M4 chip. Not a port, not an adaptation -- native.
Apple built MLX to use the unified memory architecture of their chips. Models access CPU and GPU memory without copying. Very fast.

Best for: Mac users who want maximum local performance.

WHEN TO USE LOCAL vs CLOUD:
LOCAL when:
  Privacy: data cannot leave the device (healthcare, legal, personal)
  Cost: high volume where API costs exceed GPU hardware cost
  Offline: no internet available (field work, military, remote)
  Latency: local is often faster for small models (no network round trip)
  Development: quick iteration without API rate limits

CLOUD when:
  Quality: need GPT-4o or Claude level intelligence
  Scale: need to serve thousands of users simultaneously
  Convenience: no GPU hardware to manage

QUALITY REALITY:
Local 7B model: good for simple tasks, formatting, basic Q&A.
Local 70B model: competitive with GPT-3.5 for many tasks.
Cloud GPT-4o: still significantly better for complex reasoning.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of local LLMs like having a personal library vs going to a university library. Your personal library is always available,
private, and free. The university library has more books and better resources but requires travel and a library card.

1. Ollama: the starting point for everyone. One command. Works on Mac, Linux, Windows.
2. GGUF Q4_K_M: the sweet spot quantization for local models. Best balance of quality and size.
3. Apple Silicon: M1 Pro or better with 32GB+ RAM is the sweet spot. Can run 13B models smoothly.
4. The bigger the model, the better -- but it must FIT in your RAM. A 70B model needs 40GB+ RAM.
```

5. Use local for DEVELOPMENT and TESTING. Switch to cloud for production quality.
6. ollama serve: exposes OpenAI-compatible API. Your code that calls OpenAI works with local models by changing the base URL.

WAR STORY: Developer used GPT-4 API for code assistance during development. $200/month in API costs, plus rate limit frustrations. Switched to Ollama + CodeLlama-34B on a Mac M2 Max. Zero API costs. No rate limits. Latency: 800ms instead of 2 seconds. Quality: good enough for code completion and simple refactoring. Still uses GPT-4 for complex architecture questions.

📋 POCKET NOTES

Ollama: one command. Download and run. 50+ models. ollama run llama3
llama.cpp: C++, max efficiency. CPU + Apple Metal. GGUF format.
MLX: Apple Silicon native. Fastest on Mac M1/M2/M3/M4.
Local when: privacy, cost, offline, development, latency
Cloud when: max quality, scale, convenience
Quality: 7B=basic | 70B=GPT-3.5 level | cloud GPT-4=best reasoning
GGUF Q4_K_M: sweet spot quantization for local
Apple Silicon: M1 Pro+ with 32GB RAM = 13B models comfortably
ollama serve: OpenAI-compatible API on localhost
Dev locally, deploy to cloud for production.

🎯 INTERVIEW PREP

SAY THIS: 'I use local LLMs for development, privacy-sensitive processing, and offline scenarios. Ollama for quick setup, llama.cpp for CPU-only environments. For production serving, I benchmark local vs cloud and choose based on quality requirements and cost-at-scale.'

Q: Can local models replace cloud APIs?
A: 'For specific well-defined tasks (classification, extraction, formatting): often yes. A fine-tuned local 8B model can match GPT-3.5 on narrow tasks. For general reasoning and complex tasks: cloud models are still significantly better.'

Q: Hardware recommendations?
A: 'Development: Mac with 32GB RAM runs 13B models well. Production self-hosted: 1-2 A100 GPUs for 70B quantized. Edge/mobile: 1-3B models with llama.cpp.'

⌨️ CODE

Python

```python
# Local LLM options
import subprocess

print("=== LOCAL LLM QUICK START ===")
print()
print("1. OLLAMA (easiest):")
print("   # Install: curl -fsSL https://ollama.com/install.sh | sh")
print("   # Run: ollama run llama3")
print("   # API: ollama serve  (port 11434)")
print()

# Use Ollama with standard OpenAI client
print("2. USE WITH OPENAI CLIENT (zero code changes):")
print("""   from openai import OpenAI
   client = OpenAI(
       base_url='http://localhost:11434/v1',
       api_key='ollama'  # Required but not used
   )
   response = client.chat.completions.create(
       model='llama3',
       messages=[{'role':'user','content':'Hello!'}]
   )
   # Same code as cloud OpenAI. Just different base_url.""")
print()

print("3. MODEL SIZE vs RAM:")
models = [
    ("Phi-3-mini (3.8B)", "4GB", "Basic tasks, fast"),
    ("Llama-3-8B", "8GB", "Good general purpose"),
    ("Llama-3-13B", "16GB", "Better quality"),
    ("Mixtral 8x7B", "32GB", "Near GPT-3.5 quality"),
```

```
    ("Llama-3-70B (Q4)", "40GB", "Best local quality"),
]
for model, ram, note in models:
    print(f"   {model:22s} | RAM: {ram:5s} | {note}")

print("\
4. COST COMPARISON (1M queries/month):")
print("   GPT-4o API:     ~$15,000/month")
print("   GPT-3.5 API:    ~$500/month")
print("   Local Llama-70B: ~$300/month (GPU rental)")
print("   Local Llama-8B:  ~$0/month (your laptop)")
```

📺 **OUTPUT**

```
=== LOCAL LLM QUICK START ===

1. OLLAMA: curl install, ollama run llama3
2. Same OpenAI client code. Just change base_url.
3. MODEL SIZE vs RAM:
   Phi-3-mini (3.8B)      | RAM: 4GB   | Basic tasks, fast
   Llama-3-8B              | RAM: 8GB   | Good general purpose
   Llama-3-13B             | RAM: 16GB  | Better quality
   Mixtral 8x7B            | RAM: 32GB  | Near GPT-3.5 quality
   Llama-3-70B (Q4)        | RAM: 40GB  | Best local quality

4. COST: Local = $0-300/mo vs Cloud = $500-15,000/mo
```

LLMOps

Prompt versioning, experiment tracking, model registry, CI/CD

> Imagine DevOps but for AI. DevOps automates software delivery: code -> test -> deploy -> monitor. LLMOps automates AI delivery: prompts -> evaluate -> deploy -> monitor. Every prompt is versioned like code. Every model change passes tests. Every deployment is tracked. This is what separates a prototype from a production system.

8. Full LLMOps Pipeline

📖 DEEP TEACH

```
THE FULL PIPELINE (DevOps but for LLMs):

Imagine a factory assembly line for AI. Each station has quality checks. Nothing ships without passing every check.

1. PROMPT VERSIONING (like Git for code):
Every prompt lives in Git. Changes go through pull requests with review.
  prompts/
    refund-classifier/
      v1.txt (accuracy: 91%)
      v2.txt (accuracy: 94%)
      v3.txt (accuracy: 93% -- regression! do not deploy)

2. EXPERIMENT TRACKING (like a lab notebook):
Every experiment is logged: which prompt, which model, which data, what metrics.
Tools: Weights and Biases, MLflow, LangSmith.
You can answer: 'What changed between the version that worked and the one that broke?'

3. MODEL REGISTRY (like a software release catalog):
A central registry of all models and their status:
  llama-3-8b-refund-v2: PRODUCTION (accuracy 94%)
  llama-3-8b-refund-v3: STAGING (testing)
  llama-3-8b-refund-v1: ARCHIVED

4. EVALUATION SUITE (automated quality checks):
Golden dataset of 200+ test cases. Every change must pass ALL tests.
CI/CD gate: if accuracy drops more than 2%, deploy is BLOCKED.

5. DEPLOYMENT (canary releases):
  10% of traffic to new version for 24 hours.
  Monitor: accuracy, latency, cost, user feedback.
  If good: 50% -> 100%.
  If bad: instant rollback.

6. MONITORING (production health):
  Dashboard: latency p50/p95/p99, error rate, cost/day, quality score.
  Alerts: latency > 5s, errors > 2%, cost > budget.
  Weekly: sample 1% of traffic, score with LLM-as-judge.

THE COMPLETE CYCLE:
Develop prompt -> Evaluate on golden dataset -> Pass review -> Canary deploy (10%) -> Monitor -> Full deploy (100%) -> Ongoing monitoring -> Feedback into next iteration
```

🏗 ARCHITECT TIPS + WAR STORIES

Think of LLMOps like a pharmaceutical company's process. They do not just mix chemicals and sell them. They have: version control (batch numbers), testing (clinical trials), registry (drug catalog), staged rollout (Phase 1/2/3), and post-market surveillance (monitoring).

1. TREAT PROMPTS AS CODE. Git, code review, testing, CI/CD. Non-negotiable.
2. Golden dataset grows from experience. Every production bug becomes a test case.
3. Model registry: know EXACTLY what is running in production at any time.
4. Canary deploy: NEVER deploy to 100% directly. Always 10% first.
5. The monitoring dashboard should answer: 'Is the system healthy RIGHT NOW?'

6. Weekly quality sampling catches slow degradation that real-time alerts miss.

WAR STORY: Team deployed prompts by editing them directly in production. No versioning, no tests. Someone 'improved' a prompt that broke refund classification for 3 days. $50K in incorrect decisions. Implemented LLMOps: Git for prompts, 200 test cases as CI gate, canary deploys. Next year: zero prompt-related incidents.

📋 POCKET NOTES

```
LLMOps = DevOps for AI. Automate the prompt-to-production pipeline.
6 Components:
  Prompt Versioning: Git + PR reviews. prompts/v1.txt, v2.txt.
  Experiment Tracking: W&B, MLflow, LangSmith. Log everything.
  Model Registry: catalog of models + status (prod/staging/archived).
  Evaluation Suite: 200+ golden tests. CI/CD gate. Block if quality drops.
  Deployment: canary 10% -> monitor 24h -> 50% -> 100%. Instant rollback.
  Monitoring: dashboard (latency, errors, cost). Weekly 1% quality sampling.
Cycle: develop -> evaluate -> review -> canary -> monitor -> full deploy
Treat prompts as code. Non-negotiable.
Every bug becomes a test case. Suite grows from experience.
```

🎯 INTERVIEW PREP

SAY THIS: 'I implement full LLMOps: prompts versioned in Git with code review, 200+ test golden dataset as CI/CD gate, experiment tracking in LangSmith, canary deploys with automatic rollback, and production monitoring with weekly quality sampling.'

Q: How do you prevent bad prompt deployments?
A: 'Three gates: peer review (human check), automated eval suite (200+ test cases must pass), and canary deploy (10% traffic for 24 hours with monitoring). All three must pass before full rollout.'

Q: What is on your LLMOps dashboard?
A: 'Five panels: request volume + errors, latency percentiles, token usage + cost against budget, model version distribution, and quality score trend from weekly LLM-as-judge sampling.'

⌨️ CODE

Python

```python
# LLMOps Pipeline
import hashlib, json
from datetime import datetime

class LLMOps:
    def __init__(self):
        self.prompts = {}  # version -> {text, metrics, status}
        self.experiments = []
        self.registry = {}

    def version_prompt(self, name, text, author):
        h = hashlib.md5(text.encode()).hexdigest()[:8]
        v = len([k for k in self.prompts if k.startswith(name)]) + 1
        key = f"{name}/v{v}"
        self.prompts[key] = {"text":text,"hash":h,"author":author,
            "created":datetime.now().isoformat(),"status":"staging"}
        print(f"  Versioned: {key} [{h}] by {author}")
        return key

    def evaluate(self, prompt_key, golden_tests, llm_func):
        prompt = self.prompts[prompt_key]["text"]
        passed = sum(1 for t in golden_tests
            if t["expected"].lower() in llm_func(prompt, t["input"]).lower())
        accuracy = passed / len(golden_tests)
        self.prompts[prompt_key]["accuracy"] = accuracy
        status = "PASS" if accuracy >= 0.90 else "FAIL"
        print(f"  Eval {prompt_key}: {passed}/{len(golden_tests)} = {accuracy:.0%} [{status}]")
        return accuracy >= 0.90

    def deploy(self, prompt_key, pct=10):
        if self.prompts[prompt_key].get("accuracy",0) < 0.90:
            print(f"  BLOCKED: {prompt_key} failed eval. Cannot deploy.")
```

```
            return False
        self.prompts[prompt_key]["status"] = f"canary-{pct}%"
        print(f"  Deployed: {prompt_key} to {pct}% traffic")
        return True

ops = LLMOps()

# Version prompts
v1 = ops.version_prompt("classify","Classify billing/tech: {q}","alice")
v2 = ops.version_prompt("classify","Classify billing/tech/general: {q}. One word.","bob")

# Evaluate
tests = [{"input":"How much?","expected":"billing"},
         {"input":"API error","expected":"tech"},
         {"input":"Hello","expected":"general"}]

def mock_llm(prompt, inp): return "billing" if "much" in inp else "tech" if "error" in inp else "general"

ops.evaluate(v1, tests, mock_llm)
ops.evaluate(v2, tests, mock_llm)

# Deploy
ops.deploy(v2, pct=10)
print("\
  Pipeline: version -> evaluate -> canary deploy -> monitor -> full deploy")
```

OUTPUT

```
Versioned: classify/v1 [a3f2b1c9] by alice
Versioned: classify/v2 [d7e4f5a2] by bob
Eval classify/v1: 3/3 = 100% [PASS]
Eval classify/v2: 3/3 = 100% [PASS]
Deployed: classify/v2 to 10% traffic

Pipeline: version -> evaluate -> canary deploy -> monitor -> full deploy
# Every change tracked. Every version tested. Safe deployments.
```

LIBRARIES

fastmcp	Build MCP servers	`pip install fastmcp`
ollama	Local model runner	`ollama.com/download`
llama-cpp-python	CPU/Metal inference	`pip install llama-cpp-python`
mlx-lm	Apple Silicon native	`pip install mlx-lm`
instructor	Structured output	`pip install instructor`
outlines	Constrained decoding	`pip install outlines`
mlflow	Experiment tracking	`pip install mlflow`
graphrag	Knowledge graph RAG	`pip install graphrag`

ARCHITECTURE DIAGRAM

Renders on GitHub — or paste into mermaid.live

```
graph TB
    User([User / App / API]) --> MCP_Client

    subgraph MCP [MCP — Universal Tool Protocol]
        MCP_Client[MCP Client<br/>Discovers + calls tools] --> MCP_Server1[MCP Server: DB<br/>search, read, write]
        MCP_Client --> MCP_Server2[MCP Server: APIs<br/>Slack, Drive, GitHub]
        MCP_Client --> MCP_Server3[MCP Server: Custom<br/>Your internal tools]
    end

    MCP_Client --> Gateway

    subgraph CompoundAI [Compound AI System]
        Gateway[LiteLLM Gateway<br/>Multi-provider + fallback]

        Gateway --> SemanticCache{Semantic Cache<br/>30-40% hit rate}
        SemanticCache -->|HIT| Response
```

```
        SemanticCache -->|MISS| Router

        Router[Complexity Router<br/>rules / classifier] -->|Simple 70%| CheapPath
        Router -->|Complex 30%| ExpensivePath
        Router -->|Multi-hop| GraphPath

        subgraph CheapPath [Simple Path — $0.001/query]
            BM25[BM25 Retriever<br/>FREE] --> Mini[GPT-4o-mini]
        end

        subgraph ExpensivePath [Complex Path — $0.03/query]
            Hybrid[Hybrid Search<br/>Dense + BM25 + RRF] --> Rerank[Reranker<br/>+10-20% precision]
            Rerank --> GPT4[GPT-4o]
        end

        subgraph GraphPath [Knowledge Graph Path]
            EntityQ[Entity Detection] --> Neo4j[(Neo4j Graph DB)]
            Neo4j --> Traverse[Multi-hop Traversal<br/>Bob → Alice → Carol]
            Traverse --> GPT4_G[GPT-4o + Graph Context]
        end
    end

    Mini --> Guardrails
    GPT4 --> Guardrails
    GPT4_G --> Guardrails

    subgraph Safety [Safety Layer]
        Guardrails[Input: Moderation + PII Redact + Injection Defense]
        Guardrails --> OutputCheck[Output: PII Scan + Faithfulness + Policy]
    end

    OutputCheck --> Response([Response])

    subgraph LLMOps [LLMOps Pipeline]
        direction TB
        PromptGit[Prompts in Git<br/>PR review] --> GoldenTest[Golden Dataset<br/>200+ tests]
        GoldenTest -->|Pass| CanaryDeploy[Canary 10%<br/>24h monitor]
        GoldenTest -->|Fail| Block[Block Deploy]
        CanaryDeploy -->|Healthy| FullDeploy[100% Rollout]
        CanaryDeploy -->|Degraded| Rollback[Instant Rollback]
    end

    subgraph Monitoring [Production Monitoring]
        Latency[Latency p50/p95/p99]
        Errors[Error Rate < 2%]
        Cost[Cost vs Budget]
        Faith[Faithfulness > 90%<br/>Weekly 1% sampling]
        Bias[Monthly Bias Audit]
    end

    Response --> Monitoring

    subgraph DataLayer [Data Architecture]
        VectorDB[(Qdrant<br/>Embeddings)]
        Postgres[(PostgreSQL<br/>Structured data)]
        S3[(S3 / MongoDB<br/>Raw documents)]

        subgraph EmbeddingPipeline [Embedding Pipeline]
            Extract[Extract] --> Chunk[Chunk<br/>Recursive + Parent-Child]
            Chunk --> Dedup[Dedup<br/>Content hash]
            Dedup --> EmbedStore[Embed + Store<br/>with metadata]
        end

        EmbedStore --> VectorDB
    end

    subgraph LocalLLM [Local / Edge Option]
        Ollama[Ollama<br/>ollama run llama3]
        LlamaCpp[llama.cpp<br/>CPU inference]
        MLX_Local[MLX<br/>Apple Silicon]
    end

    Gateway -.->|Privacy / offline<br/>/ development| LocalLLM

    subgraph Distillation [Model Optimization]
        Teacher[GPT-4 labels 10K examples<br/>$300] --> Student[Fine-tune Llama-8B<br/>$50]
        Student --> Quantize[Quantize to 4-bit<br/>100x cheaper than GPT-4]
        Quantize --> vLLM[Deploy via vLLM<br/>2-4x throughput]
    end

    subgraph MoE [Mixture of Experts]
        Mixtral[Mixtral 8x7B<br/>47B knowledge<br/>13B compute per token<br/>Best self-hosting value]
    end

    vLLM --> Gateway
    Mixtral --> Gateway

    style Gateway fill:#1f6feb,color:#fff
    style SemanticCache fill:#d29922,color:#fff
    style Router fill:#8957e5,color:#fff
    style Guardrails fill:#da3633,color:#fff
    style Response fill:#3fb950,color:#fff
    style MCP_Client fill:#1f6feb,color:#fff
    style FullDeploy fill:#3fb950,color:#fff
    style Block fill:#da3633,color:#fff
    style Faith fill:#da3633,color:#fff
```

— POCKET NOTES —

1. MCP Servers, Clients, and Building Integrations

```
MCP = USB-C for AI tools. ONE standard protocol for all tool connections.
Server: wraps a tool, exposes standard interface (search, read, create)
Client: LLM side. Discovers and calls any MCP server.
Host: orchestrator (Claude Desktop, VS Code, custom app)
Before MCP: 10 tools x 5 LLMs = 50 custom integrations
With MCP: 10 servers + 5 clients = 15 standard pieces
FastMCP (Python): build server in 20 lines. pip install fastmcp.
Hundreds of pre-built servers exist (Drive, Slack, GitHub, DBs)
Security: validate inputs, authenticate callers.
Open standard by Anthropic. Being adopted industry-wide.
```

2. Multi-Component Architecture + Orchestration

```
Compound AI = multiple specialized components working together.
Analogy: hospital with specialists, not one doctor for everything.
Typical components: classifier + retriever + generator + guardrails + cache
Why not one model? Cost (30x), speed (20ms vs 2s), control, reliability.
Orchestrator: LangGraph. Each component = graph node.
Principles: single responsibility, loose coupling, observable, graceful degradation.
Cost: compound system typically 80-90% cheaper than one big model.
Start simple (retriever + generator). Add components for specific pain points.
Each component independently testable and deployable.
```

3. GraphRAG + Entity Extraction + When to Use

```
Knowledge Graph: entities (nodes) + relationships (edges)
GraphRAG: extract entities from docs -> build graph -> traverse for multi-hop Qs
Problem it solves: 'Bob reports to Alice reports to Carol' across 3 documents
Regular RAG finds individual chunks. GraphRAG follows chains.
4 steps: entity extraction -> graph construction -> community detection -> query
When: 20%+ queries need cross-document connections
When NOT: simple factual lookups, independent documents
Cost: ~$2K per 100K documents for entity extraction
Neo4j: standard graph DB. Microsoft graphrag library.
Hybrid: regular RAG + GraphRAG routed by query type.
```

4. Compression + Recursive Summarization + Memory

```
3 Techniques for exceeding context windows:
  Compression (LLMLingua): remove filler words. 10K -> 3K words. 90% quality.
  Recursive Summarization: summarize chapters -> summarize summaries -> final.
  Memory: sliding window, memory tokens, retrieval-augmented.
Practical hierarchy:
  1. Fits in window? Use directly.
  2. Too big? RAG (retrieve relevant chunks only).
  3. Need whole doc? Recursive summarization (MapReduce).
  4. Need detail? Compression (LLMLingua).
Lost in the middle: put important info at START and END of context.
128K tokens = ~300 pages. Test if you actually exceed it.
pip install llmlingua
```

5. Constrained Decoding + Knowledge Distillation

```
Structured Generation (3 levels):
  JSON mode: valid syntax, no schema guarantee.
  Instructor + Pydantic: exact schema + auto-retry. INDUSTRY STANDARD.
  Outlines/Guidance: token-level control. Guaranteed. Local models only. Faster.
Distillation:
  GPT-4 generates 10K examples -> fine-tune small model -> deploy at 10-50x less cost.
```

```
  Quality: 85-95% of teacher. Cost: 10-50x cheaper.
  When: high volume (100K+/day), well-defined task, quality gap acceptable.
  When NOT: creative tasks, low volume, frequently changing tasks.
Legal: check ToS for using outputs to train competing models.
Combo: distill + quantize = 100x cheaper than GPT-4.
```

6. How MoE Works + Mixtral + Practical Implications

```
MoE: 8 expert sub-networks + router. Only 2 experts active per token.
Mixtral 8x7B: 47B total params, 13B active. Big knowledge, small speed cost.
Router: small network picks top 2 experts per token.
Memory: ALL experts loaded (47B). Speed: only 2 compute (13B).
Best value for self-hosting: big-model quality at small-model speed.
Models: Mixtral 8x7B, 8x22B, DeepSeek-V2, DBRX.
Fine-tuning MoE: harder (expert routing can unbalance).
vLLM: handles MoE routing efficiently.
```

7. Ollama + llama.cpp + MLX + When to Use

```
Ollama: one command. Download and run. 50+ models. ollama run llama3
llama.cpp: C++, max efficiency. CPU + Apple Metal. GGUF format.
MLX: Apple Silicon native. Fastest on Mac M1/M2/M3/M4.
Local when: privacy, cost, offline, development, latency
Cloud when: max quality, scale, convenience
Quality: 7B=basic | 70B=GPT-3.5 level | cloud GPT-4=best reasoning
GGUF Q4_K_M: sweet spot quantization for local
Apple Silicon: M1 Pro+ with 32GB RAM = 13B models comfortably
ollama serve: OpenAI-compatible API on localhost
Dev locally, deploy to cloud for production.
```

8. Full LLMOps Pipeline

```
LLMOps = DevOps for AI. Automate the prompt-to-production pipeline.
6 Components:
  Prompt Versioning: Git + PR reviews. prompts/v1.txt, v2.txt.
  Experiment Tracking: W&B, MLflow, LangSmith. Log everything.
  Model Registry: catalog of models + status (prod/staging/archived).
  Evaluation Suite: 200+ golden tests. CI/CD gate. Block if quality drops.
  Deployment: canary 10% -> monitor 24h -> 50% -> 100%. Instant rollback.
  Monitoring: dashboard (latency, errors, cost). Weekly 1% quality sampling.
Cycle: develop -> evaluate -> review -> canary -> monitor -> full deploy
Treat prompts as code. Non-negotiable.
Every bug becomes a test case. Suite grows from experience.
```

— INTERVIEW PREP —

1. MCP Servers, Clients, and Building Integrations

```
SAY THIS: 'I build tool integrations as MCP servers using FastMCP. This gives a standard interface that works with any LLM client --
Claude, GPT-4, or local models. One implementation serves all platforms.'

Q: Why MCP instead of custom tool calling?
A: 'Portability. A custom OpenAI function call only works with OpenAI. An MCP server works with any MCP-compatible client. Build once,
connect everywhere. Plus the ecosystem has hundreds of pre-built servers.'

Q: How do you secure MCP servers?
A: 'Authentication tokens per client, input validation on all parameters, rate limiting, and audit logging. An MCP server is an API --
apply the same security practices as any production API.'
```

2. Multi-Component Architecture + Orchestration

SAY THIS: 'I design compound AI systems where each component is the best tool for its job. A fine-tuned BERT classifier for routing, BM25 + dense hybrid for retrieval, GPT-4 only for complex generation. This typically reduces cost 80-90% while maintaining or improving quality.'

Q: Why not just use GPT-4 for everything?
A: 'Three reasons: cost (GPT-4 classification costs 30x more than BERT), latency (BERT: 20ms, GPT-4: 2s for the same task), and reliability (each component fails independently, enabling graceful degradation).'

Q: How do you orchestrate?
A: 'LangGraph state machine. Each component is a node with conditional routing. Full pipeline tracing for observability. Each component is independently deployable and testable.'

3. GraphRAG + Entity Extraction + When to Use

SAY THIS: 'I use GraphRAG when queries require multi-hop reasoning across documents -- connecting entities through relationships that span multiple sources. I route: simple factual queries to standard RAG, relationship queries to GraphRAG.'

Q: When is GraphRAG worth the investment?
A: 'When more than 20% of queries need information connected across documents. Entity extraction costs ~$2K per 100K docs. I pilot on 1000 documents first to validate the use case.'

Q: How do you build the graph?
A: 'LLM extracts entities and relationships from each document. Store in Neo4j. Community detection clusters related entities. Microsoft graphrag library automates most of this pipeline.'

4. Compression + Recursive Summarization + Memory

SAY THIS: 'I follow a practical hierarchy: direct context if it fits, RAG for targeted retrieval, recursive summarization for full-document processing, and compression for maximizing information density. 90% of the time, RAG makes the long-context problem disappear.'

Q: 2000-page legal document. How do you process it?
A: 'Two-pass: recursive summarization for a 10-page overview that catches all major issues, then RAG for specific clause-level questions. Neither alone is sufficient -- the summary gives breadth, RAG gives depth.'

Q: Lost in the middle -- what is it and how do you handle it?
A: 'Models attend more to the beginning and end of context, less to the middle. I put the most important information at the start and end. For retrieved chunks, I order by relevance with the best match first.'

5. Constrained Decoding + Knowledge Distillation

SAY THIS: 'I use instructor for production structured output -- Pydantic schemas with auto-retry. For local models, Outlines gives token-level guarantees. For high-volume tasks, I distill GPT-4 into smaller models to cut inference cost 10-50x while retaining 90%+ quality.'

Q: How do you decide what to distill?
A: 'Three criteria: volume above 100K calls/day (cost savings justify training), well-defined task (classification, extraction -- not open-ended creativity), and 85-95% quality is acceptable for the use case.'

Q: Is distillation legal?
A: 'Depends on the provider ToS. OpenAI allows it for certain models. Anthropic has specific terms. Always check. Alternatively, distill from open-source teachers (Llama-70B teaching Llama-8B) to avoid any legal concerns.'

6. How MoE Works + Mixtral + Practical Implications

SAY THIS: 'MoE models like Mixtral give the knowledge of a 47B model at the inference cost of a 13B model. The router activates only 2 of 8 experts per token. For self-hosting, this is the best quality-per-dollar ratio.'

Q: Why not just use a 13B dense model?
A: 'A dense 13B model has 13B of knowledge. Mixtral has 47B of knowledge but only uses 13B of compute. The MoE model is significantly smarter at the same speed. Different experts specialize in different types of content.'

Q: Downsides of MoE?
A: 'Memory: all experts must be loaded (47B for Mixtral). Fine-tuning is harder because expert balance can degrade. And the routing adds slight overhead.'

7. Ollama + llama.cpp + MLX + When to Use

SAY THIS: 'I use local LLMs for development, privacy-sensitive processing, and offline scenarios. Ollama for quick setup, llama.cpp for CPU-only environments. For production serving, I benchmark local vs cloud and choose based on quality requirements and cost-at-scale.'

Q: Can local models replace cloud APIs?
A: 'For specific well-defined tasks (classification, extraction, formatting): often yes. A fine-tuned local 8B model can match GPT-3.5 on narrow tasks. For general reasoning and complex tasks: cloud models are still significantly better.'

Q: Hardware recommendations?
A: 'Development: Mac with 32GB RAM runs 13B models well. Production self-hosted: 1-2 A100 GPUs for 70B quantized. Edge/mobile: 1-3B models with llama.cpp.'

8. Full LLMOps Pipeline

SAY THIS: 'I implement full LLMOps: prompts versioned in Git with code review, 200+ test golden dataset as CI/CD gate, experiment tracking in LangSmith, canary deploys with automatic rollback, and production monitoring with weekly quality sampling.'

Q: How do you prevent bad prompt deployments?
A: 'Three gates: peer review (human check), automated eval suite (200+ test cases must pass), and canary deploy (10% traffic for 24 hours with monitoring). All three must pass before full rollout.'

Q: What is on your LLMOps dashboard?
A: 'Five panels: request volume + errors, latency percentiles, token usage + cost against budget, model version distribution, and quality score trend from weekly LLM-as-judge sampling.'